RAG System Report

Retrieval evaluation (BM25 / Word2Vec / Hybrid) and Llama 3.2 generation over a crawled software-documentation corpus

I. System report

1. System design + Corpus construction

a. General Architecture

Please refer to **Figure 1** for the architecture of the system.

The system consists of several major components: a web crawler, data preprocessing pipeline, document and chunk storage, retrieval models, and an evaluation component.

The crawler collects documentation from selected software development websites. The collected documents are then cleaned, normalized, deduplicated, and divided into smaller chunks. These chunks are used to build the retrieval models, including BM25 and Word2Vec-based retrieval. A hybrid retrieval model combines the results from both retrieval approaches to improve retrieval performance. Finally, the most relevant chunks are passed to the Llama language model to generate an answer based only on the retrieved context. The retrieval results are also evaluated using a manually constructed evaluation set.

b. Crawler

The crawler is responsible for collecting documentation from selected software development websites. The crawler starts from a predefined set of seed URLs and follows links within the allowed websites to discover additional documentation pages. Before accessing a URL, the crawler checks the website's robots.txt rules to determine whether the page is permitted to be crawled.

The crawler uses HTTP requests to retrieve web pages and BeautifulSoup to parse their HTML content. It also handles different content types, including HTML and Markdown-based documentation. For Markdown content, the crawler converts the content into HTML before processing it. Invalid pages, such as error pages, index pages, and unsupported content types, are filtered out.

To reduce redundant data, the crawler performs deduplication using SHA-256 hashes. A hash is calculated for the document content and title, allowing duplicate documents or pages with identical content to be skipped. The crawler stores the collected documents together with metadata such as the document number, title, URL, and extracted content.

c. Data processor

The data processor transforms the raw documents collected by the crawler into a format suitable for information retrieval. First, unwanted HTML elements, such as scripts and other non-content elements, are removed. The remaining text is extracted and normalized to reduce unnecessary formatting differences.

The processor also performs document-level deduplication and divides documents into smaller overlapping chunks. Chunking allows long documentation pages to be represented as smaller units of information, which improves the ability of the retrieval models to identify the specific portion of a document that is relevant to a query.

Each chunk is stored with associated metadata, including its document identifier, title, URL, and chunk text. The resulting collection of chunks is then used as the input corpus for both BM25 and Word2Vec retrieval.

d. BM25 model

The BM25 model is used as the primary lexical retrieval method. BM25 ranks chunks based on the occurrence of query terms within each chunk while considering factors such as term frequency, inverse document frequency, and document length.

The implementation uses the BM25Okapi algorithm from the rank_bm25 library. The tokenized chunks are provided as the corpus when constructing the BM25 index:

```
bm25 = BM25Okapi(corpus)
```

For a given user query, the query is tokenized using the same preprocessing procedure as the document chunks. BM25 then calculates a relevance score for every chunk. Chunks with higher BM25 scores are considered more relevant to the query and are ranked higher in the retrieval results.

BM25 is particularly effective for software documentation because it can directly match technical terms, API names, class names, configuration properties, and other keywords appearing in both the query and documentation.

e. Word2Vec model

The Word2Vec model provides a semantic retrieval approach that complements the lexical matching performed by BM25. Word2Vec learns vector representations of words based on their surrounding context in the documentation corpus. Words that occur in similar contexts are represented by similar vectors.

The system trains Word2Vec using the Skip-gram (sg=1) architecture. The model uses a vector size defined by vector_size, a context window of 10 words, a minimum word frequency of 1, 10 worker threads, 10 negative samples, and 20 training epochs. A fixed random seed of 42 is used to make the training process reproducible.

After building the vocabulary, the model is trained using the processed chunk sentences:

```
model.build_vocab(sentences) model.train(sentences,
total_examples=model.corpus_count, epochs=model.epochs)
```

The trained model is then saved for later retrieval.

To represent an entire chunk as a single vector, the system obtains the Word2Vec vector for each token in the chunk. Instead of simply averaging all word vectors equally, the system uses TF-IDF weights to give more importance to informative words. The resulting TF-IDF-weighted average produces a vector representation for each chunk.

This approach allows the retrieval system to identify chunks that are semantically related to the query even when they do not contain exactly the same terms.

f. Hybrid model

The hybrid retrieval model combines the lexical retrieval capability of BM25 with the semantic retrieval capability of Word2Vec. This approach is used because the two retrieval methods have different strengths. BM25 is effective at identifying exact keyword and technical-term matches, while Word2Vec can identify semantically related terms that may not have exact lexical matches.

Since BM25 and Word2Vec produce scores on different scales, the scores from both models are first normalized using Min-Max scaling to the range [0, 1].

The hybrid score is then calculated using a weighted combination of the normalized scores:

```
hybrid_score = bm25_weight × bm25_score + (1 – bm25_weight) × w2v_score
```

In the implementation, the Word2Vec weight is calculated as:

```
vec_weight = 1.0 - bm25_weight
```

The final score is therefore:

```
score = bm25_weight * b + vec_weight * v
```

where b is the normalized BM25 score and v is the normalized Word2Vec score. The chunks are then ranked according to their hybrid scores.

This allows the system to control the contribution of lexical and semantic retrieval through the bm25_weight parameter. A higher value places more emphasis on exact keyword matching, while a lower value gives more importance to semantic similarity.

g. Llama3.2

Llama is used as the final question-answering component of the system. Rather than directly returning retrieved chunks to the user, the system passes the most relevant chunks to the Llama 3.2 language model as context.

The system uses a closed-book question-answering approach. The model is explicitly instructed to answer only using the context provided by the application and not to rely on external or pretrained knowledge. If the retrieved context does not contain enough information to answer the question, the model is instructed to respond with exactly I don't know.

The system prompt also requires every factual claim in the generated answer to be supported by a citation to one of the retrieved chunks. This constraint is intended to reduce hallucinations and ensure that generated answers are grounded in the documentation retrieved by the system.

For each query, the system selects the top-ranked retrieval results and constructs a context string. At most three chunks are provided to Llama, and each chunk is limited to 1,000 characters to reduce the amount of input context and improve response latency. Each context entry contains the document title and the corresponding chunk text.

The resulting prompt contains the user's question and the retrieved context. This information is passed to the locally running llama3.2:latest model, which generates the final answer. The model is also configured with a maximum output length of 300 tokens.

The overall process can therefore be summarized as:

```
User Query → Retrieval → Top Relevant Chunks → Context Construction → Llama → Grounded Answer
```

2. Evaluation set construction

An evaluation set was constructed to measure the effectiveness of the retrieval system. The evaluation queries were designed to represent realistic questions that a user might ask when searching through software documentation.

Each query was manually associated with its corresponding relevant document or chunk when an answer could be found within the collected corpus. Queries for which no corresponding document existed were labeled as unanswerable.

The evaluation set therefore contains both answerable and unanswerable queries. For answerable queries, the expected relevant document/chunk is used as the ground truth. The retrieved results from BM25, Word2Vec, and the hybrid model are then compared against this ground truth.

The evaluation process checks whether the correct document is retrieved and, when applicable, its ranking position. This allows the system to determine not only whether a retrieval method can find the relevant document, but also whether it can rank that document highly among the retrieved results.

Before scoring, a small number of ground-truth pages were spot-checked directly against the live documentation to confirm the evaluation set is grounded correctly:

- **Query 1:** the ground-truth page (docs.spring.io/spring-boot/reference/using/auto-configuration.html) confirms the reference answer — auto-configuration is enabled via @EnableAutoConfiguration or @SpringBootApplication, and disabled per-class via the exclude attribute. It does not describe @Conditional* annotations as the general enabling mechanism; those govern custom auto-configuration classes on a different page, a related but distinct concept.

- **Query 16:** the ground-truth page (nodejs.org/api/cluster.html) describes round-robin and shared-socket distribution of connections across forked worker processes. It contains no built-in database engine of any kind, meaning Query 16 as worded has a false premise even though a document was assigned to it as "ground truth."

3. Experiment result

Please refer to **Evalution/** folder in GitHub repository for actual retrieved result.

Sixteen queries were designed for the evaluation set; two of them (Queries 3 and 4) had no corresponding document in the crawled corpus and were correctly labeled unanswerable, leaving 14 queries with a valid ground-truth document for the retrieval comparison. Retrieval performance was measured over the top 50 results returned by each method.

Retrieval performance (top-50 window, n = 14 scoreable queries)

Retriever	Hit rate (correct doc found @50)	Mean rank when found
BM25	14/14 (100%)	7.2
W2V	11/14 (79%)	16.5
Hybrid	14/14 (100%)	8.2

BM25 is the strongest single retriever on this corpus: it never fully misses the correct document and achieves the best ranking quality of the three methods (highest MRR, lowest mean rank). This matches the design expectation in section 1.d — software documentation is dense with exact technical terms (API names, annotations, class names), which lexical matching handles directly.

W2V is a weaker retriever. It misses the correct document entirely on 3 of 14 queries (Q6, Q14, Q15 — all Node.js cluster-module questions), and even on the queries where it does find the correct document, it ranks it much lower on average (16.5 vs. 7.2 for BM25). This is consistent with the semantic-similarity approach in section 1.e being a weaker fit for terse, jargon-heavy API documentation, where two chunks can be topically adjacent (e.g., cluster.html vs. worker_threads.html vs. domain.html) without one being the actually correct answer.

The hybrid retriever (section 1.f) recovers all of W2V's misses — it matches BM25's 100% hit rate — but its MRR (0.492) still sits below BM25 alone (0.598). Blending in W2V's weaker semantic scores via the weighted Min-Max combination pulls some correct documents further down the ranking than BM25 alone would place them, even though it never causes the retriever to miss a document outright.

Per-query best rank (lower is better; — = not found in top 50)

Q	Topic	BM25	W2V	Hybrid
1	Spring auto-config	6	45	8
2	Actuator health	1	1	1
5	Node stream backpressure	22	11	23
6	Node cluster / CPU cores	12	—	20
7	Node ESM vs CJS	2	16	4
8	React state reset	1	25	1
9	React render steps	2	1	2

Q	Topic	BM25	W2V	Hybrid
10	React Suspense	1	1	1
11	Angular DI + zoneless (multi-hop)	1	38	2
12	Spring auto-config × Actuator (multi-hop)	1	4	1
13	React state preservation × render (multi-hop)	1	20	1
14	Node stream × cluster (multi-hop)	20	—	23
15	React Context “persists after browser close”	30	—	25
16	Node cluster “database engine” (false premise)	1	19	3

A clear pattern shows up on the multi-hop queries that require two source documents (Q11–14): BM25 and Hybrid reliably rank the first target document at or near rank 1, but the second target document lands much lower or is missed outright (Q14: neither BM25 nor Hybrid surfaces the stream documentation anywhere in the top 50; only the cluster documentation is recovered). All three retrievers behave, in effect, like single-topic retrievers — strong on the dominant keyword cluster in a compound question, weak on a second, less-emphasized topic within the same query.

Generation performance (16 queries × 3 systems = 48 responses, one generation run per query)

The generation evaluation set contains 12 answerable queries (Q1, Q2, Q5–14) and 4 designed-unanswerable queries where the reference answer is “I don't know”: Q3 and Q4 ask about real Spring Framework topics (declarative-transaction self-invocation; singleton/prototype/request bean scopes) that are simply not covered by the crawled corpus, while Q15 and Q16 are false-premise questions (React Context surviving a closed browser; a “database engine” inside the Node cluster module). Each of the 48 responses was compared directly against its reference answer and coded as:

A: Correct/mostly correct,

A~: partial (attempts an answer but is incomplete, hedged, or off-target)

R: Refused (“I don't know”), or hallucinated (confidently answers a query the reference says is unanswerable).

P: Policy violation: answered using outside/pretrained knowledge instead of the retrieved context, in violation of the closed-book instruction (section 1.g)

Q	Type	BM25	W2V	Hybrid
1	Answerable	A~	A	A~
2	Answerable	A	R	A~
3	Unanswerable	A~	A~	A~
4	Unanswerable	P	A~	A~
5	Answerable	A	R	A~
6	Answerable	A	A	A
7	Answerable	A	A	A
8	Answerable	R	R	R

Q	Type	BM25	W2V	Hybrid
9	Answerable	A	A	A~
10	Answerable	A	A	A
11	Answerable	R	R	R
12	Answerable	R	R	R
13	Answerable	R	R	R
14	Answerable	A~	R	A~
15	Unanswerable	R	R	R
16	Unanswerable	R	R	R

Comparing each response to its reference answer, three findings stand out.

First, on the 12 answerable queries, systems attempted an actual answer only 61% of the time (22 of 36 responses); the shortfall is concentrated almost entirely in Q8, Q11, Q12, and Q13, which are refused by all three systems even though a reference answer exists for each.

Second, of the 22 attempted answers to answerable queries, 13 (59%) reasonably match the reference and 9 (41%) are partial or off-target — missing part of a two-part question or hedging on a detail — but essentially none invent facts outright; fabrication is concentrated on the unanswerable queries instead.

Third, on the 4 unanswerable queries, systems correctly said "I don't know" only 58% of the time (7 of 12); this splits sharply by query type — Q15 and Q16, whose false premise is stated directly in the question, are refused correctly in all 6 instances, while Q3 and Q4, which ask about real concepts simply missing from this corpus, are answered confidently and incorrectly in 5 of 6 instances.

Per-system attempt rate on answerable queries was BM25 67% (8/12), Hybrid 67% (8/12), and W2V 50% (6/12) — W2V refuses most often, consistent with it being the weakest retriever.

4. Error Analysis

- **a. Hallucination is concentrated on unanswerable queries, not answerable ones.** On Q3 and Q4 (real topics missing from the corpus), W2V and Hybrid both answer confidently using outside knowledge rather than the retrieved context — e.g. Hybrid's Q3 answer describes @Transactional as implemented via Spring AOP proxying, a plausible but unsupported claim, since the retrieved chunks are about Kafka, not transaction management. This is a direct violation of the closed-book instruction in section 1.g.
- **b. Multi-part questions get partial coverage even when answered.** Q2 ("What are Spring Boot Actuator endpoints used for, and how can you create a custom HealthIndicator?") illustrates this well: BM25 answers both parts correctly, but W2V and Hybrid each answer the first part correctly and then hedge or contradict themselves on the second ("I don't know how to create a custom HealthIndicator without more specific information") even though their own preceding sentence already described the HealthIndicator interface. The same pattern appears on Q7, where all three systems describe only some of the CommonJS/ESM detection rules.
- **c. Multi-hop and even some single-hop answerable questions are refused outright.** Q11–Q13 (compound, two-document questions) and Q8 (a single-hop React question) are refused by all three systems, consistent with the retrieval-side finding in section 3 that these queries only surface part of the needed context.

- **d. Cross-domain corpus contamination.** BM25's Q4 answer, while explaining Spring's request bean scope, pulls in an unrelated explanation of AngularJS directive scope from a different chunk in the mixed corpus, conflating two unrelated frameworks.

5. Limitation

- **Retrieval and generation evaluations don't fully agree.** Retrieval is scored at the document level (top-50 window), while generation only ever sees the top-3 chunks passed to Llama, so the two evaluations are complementary rather than directly comparable.
- **Only one generation run per query was collected.** Llama 3.2 sampling is non-deterministic, so a single transcript per query is not a reliable estimate of how a system would answer that same question in general — repeated runs would be needed to establish a per-query success rate.
- **Response coding is manual and somewhat subjective.** Each of the 48 responses was hand-labeled by comparison to its reference answer; no automated faithfulness or citation-verification metric was used, and partial answers required a judgment call.
- **The evaluation corpus and query set have some noise.** The corpus mixes several unrelated tech ecosystems, and at least one benchmark question (Q16) has a false premise in its ground-truth document, both of which limit how cleanly the results can be interpreted.

II. GitHub repository

URL: https://github.com/fanggodzzz/Full-Stack-RAG

III. Demo

URL: https://drive.google.com/drive/folders/1lMx9Gckbl-TI3OtiOMskdIR-DHKVpE6v?usp=sharing

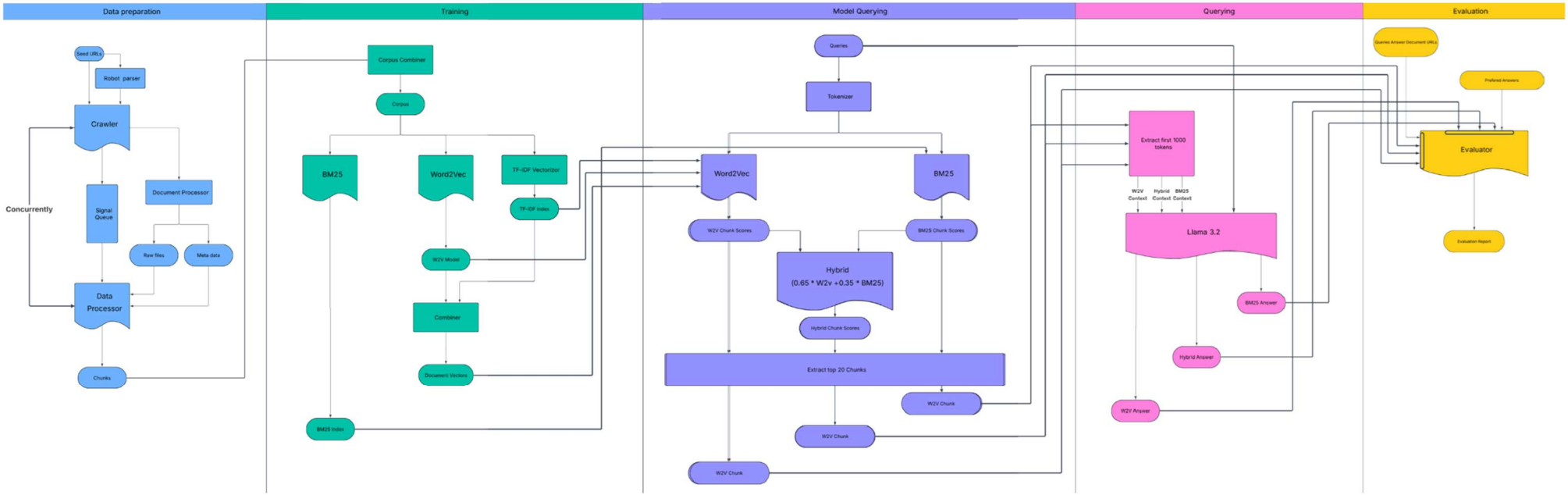
Data preparation
Training
Model Querying
Querying
Evaluation
Seed URLs
Robot parser
Crawler
Concurrently
Signal Queue
Document Processor
Raw files
Meta data
Data Processor
Chunks
Corpus Combiner
Corpus
BM25
Word2Vec
TF-IDF Vectorizor
TF-IDF Index
W2V Model
Combiner
Document Vectors
BM25 Index
Queries
Tokenizer
Word2Vec
BM25
W2V Chunk Scores
BM25 Chunk Scores
Hybrid
(0.65 * W2v +0.35 * BM25)
Hybrid Chunk Scores
Extract top 20 Chunks
W2V Chunk
W2V Chunk
W2V Chunk
Extract first 1000 tokens
W2V Context
Hybrid Context
BM25 Context
Llama 3.2
BM25 Answer
Hybrid Answer
W2V Answer
Queries Answer Document URLs
Preferred Answers
Evaluator
Evaluation Report

Figure 1